

CPE-414
Embedded System

Smart Home Adaptive Control
System

จัดทำโดย

นายธีระพงศ์	สุขขีนิยง	รหัสนักศึกษา	2311311126
นายณนัท	เจริญสุข	รหัสนักศึกษา	2311311555

นำเสนอ

อาจารย์จิตติชญา ธนมิตรสมบูรณ์

Executive Summary

รายงานฉบับนี้อธิบายระบบ **Smart Home Adaptive Control System** เวอร์ชันที่พัฒนาและใช้งานจริงในโครงการ โดยระบบปัจจุบันเป็นต้นแบบด้านความปลอดภัยภายในบ้านที่ใช้บอร์ด `'ESP32-WROOM-32'` เพียงตัวเดียวเป็นแกนหลัก หน้าที่ของระบบคือเฝ้าระวังการบุกรุก ควบคุมการล็อกประตูและหน้าต่าง ให้ผู้ใช้โต้ตอบผ่าน **keypad** และ **OLED** และเชื่อมต่อออกสู่ภายนอกผ่าน **MQTT** และ **LINE OA** เพื่อใช้แจ้งเตือนและสั่งงานระยะไกล

ฮาร์ดแวร์หลักที่ใช้ประกอบด้วยเซนเซอร์ `'Y213'` สำหรับตรวจสอบสถานะประตูและหน้าต่าง, `'HC-SR501'` จำนวน 3 ตัวสำหรับตรวจจับการเคลื่อนไหวในแต่ละโซน, `'SW-1801P'` สำหรับตรวจจับแรงสั่นสะเทือนหรือการกดแฉะ, `'HC-SR04'` จำนวน 3 ชุดสำหรับตรวจจับจุด chokepoint, keypad แบบ `'4x4 Matrix Membrane Keypad (#27899)'` ผ่าน `'PCF8574 I2C Expander'`, จอ `'SSD1306 OLED'`, เซอร์โว `'SG90'` สำหรับล็อกประตูและหน้าต่าง และ **piezo buzzer** สำหรับเตือนสถานะ **Warn** และ **Alert**

ซอฟต์แวร์บนบอร์ดทำงานบน **FreeRTOS** โดยแยกเป็นสอง **task** ชัดเจน คือ `'SecTask'` รอบเวลา 20 ms สำหรับจัดการตรรกะความปลอดภัย และ `'MqttTask'` รอบเวลา 10 ms สำหรับจัดการเครือข่ายและ **MQTT** การแยก **task** ลักษณะนี้ทำให้การ **reconnect** เครือข่ายไม่ไปบล็อกความปลอดภัยหลัก ส่งผลให้ระบบยังตอบสนองต่อ keypad ปุ่มกดจริง และเซนเซอร์ได้ต่อเนื่องแม้เครือข่ายมีปัญหา ข้อมูลสำคัญ เช่น `'latest_mode'` และ `'is_night'` ถูกบันทึกลง **NVS** เพื่อให้ระบบกลับมาทำงานได้อย่างสอดคล้องหลังการรีสตาร์ทหรือไฟดับ

Introduction

ระบบความปลอดภัยภายในบ้านแบบฝังตัวจำเป็นต้องให้ความสำคัญกับความรวดเร็ว ความเสถียร และลำดับตรรกะที่คาดเดาได้ การตรวจจับผู้บุกรุกต้องไม่พึ่งพาเพียงเซนเซอร์ตัวเดียว แต่ต้องพิจารณารูปแบบเหตุการณ์ร่วมกัน เช่น ประตูเปิดร่วมกับแรงสั่นสะเทือน การเคลื่อนไหวภายในบ้านในช่วงที่ไม่มีผู้อยู่อาศัย หรือการหมดเวลายืนยันตัวตนหลังเกิดสัญญาณ **Warn**

โครงการนี้จึงออกแบบระบบให้ผสมผสานข้อมูลจากหลายอินพุตเข้าด้วยกัน ทั้ง keypad, ปุ่มล็อกปลดล็อกจริง, reed switch, PIR, vibration sensor และ ultrasonic sensor แล้วแปลงเหตุการณ์เหล่านั้นเข้าสู่ state machine กลางเพื่อกำหนดว่าเมื่อใดควร Disarm, Warn, Alert, Auto-Arm หรือ Auto-Lock พร้อมทั้งเปิดทางให้ผู้ใช้สามารถควบคุมระบบผ่าน LINE OA โดยยังคงข้อกำหนดด้าน policy เช่น `arm\_night` ต้องมาจากรูณะ `latest\_mode == Disarm` และ `night\_off` ใช้ได้เฉพาะเมื่อระบบอยู่ใน `Night` จริง

รายงานฉบับนี้จึงมีจุดประสงค์เพื่ออัปเดตรายละเอียดทั้งหมดให้สอดคล้องกับงานที่พัฒนาสำเร็จจริง โดยอ้างอิงจากเอกสารในโฟลเดอร์ `docs` ทั้งหมด รวมถึง `README.md`, `possiblecase.md`, `layer\_structure.md`, block diagram, flowchart และ pin allocation เพื่อให้เนื้อหาในรายงานตรงกับทั้งโค้ด สถาปัตยกรรม และพฤติกรรมของระบบในเดโมจริง

Objectives

1. พัฒนาระบบรักษาความปลอดภัยภายในบ้านบน `ESP32-WROOM-32` ให้ทำงานได้จริงในระดับต้นแบบ
2. รองรับโหมดการทำงานหลัก 3 โหมด ได้แก่ `Disarm`, `Away` และ `Night`
3. ตรวจสอบการบุกรุกจากหลายแหล่งสัญญาณ เช่น ประตู หน้าต่าง การเคลื่อนไหว แรงสั่นสะเทือน และ chokepoint detection
4. รองรับการใช้งานแบบ local ผ่าน keypad, OLED และปุ่มลัดกดปลดล็อกจริงสำหรับประตูและหน้าต่าง
5. ควบคุมอุปกรณ์กระทำ ได้แก่ SG90 door servo, SG90 window servo และ piezo buzzer ตามสถานะของระบบ
6. เชื่อมต่อ MQTT และ LINE OA เพื่อให้ตรวจสอบสถานะ รับแจ้งเตือน และสั่งงานระยะไกลได้
7. ออกแบบ runtime architecture ให้เครือข่ายไม่บล็อกลูปความปลอดภัยหลัก และสามารถกู้คืนสถานะสำคัญจาก NVS ได้

System Overview

ระบบที่พัฒนาเสร็จจริงมีลักษณะเป็น **embedded security controller** ที่รวมทุกฟังก์ชันหลักไว้ในบอร์ดเดียว โดยมี ``SystemContext`` เป็นศูนย์กลางของ **state** และ ``RuleEngine`` เป็นตัวตัดสินใจเมื่อมีเหตุการณ์เกิดขึ้น ทั้งเหตุการณ์จาก **local input** และ **remote command** จะถูกรวมเข้าสู่กระบวนการที่กำหนดลำดับชัดเจน เพื่อให้สถานะ **mode**, **alarm level**, **lock state**, **latest_mode**, **is_night** และตัวแปรระหว่างทาง เช่น **exit stage** หรือ **door session** มีความสอดคล้องกันเสมอ

ในระดับภาพรวม ระบบแบ่งการทำงานออกเป็น 4 ส่วน คือ **sensing layer** สำหรับรับสัญญาณจากฮาร์ดแวร์, **processing and control layer** สำหรับแปลงเหตุการณ์และตัดสินใจ, **actuation layer** สำหรับขับ servo กับ buzzer และ **cloud communication layer** สำหรับเชื่อมกับ broker, LINE bridge และผู้ใช้ระยะไกล การเชื่อมโยงแต่ละส่วนถูกระบุไว้ใน ``docs/blockdiagram.mmd`` และ ``docs/flowchart.mmd`` และรายงานฉบับนี้ใช้เอกสารดังกล่าวเป็นแหล่งอ้างอิงหลัก

Security & Communication Core (ESP32-WROOM-32)

แกนหลักของระบบคือ ``ESP32-WROOM-32`` ซึ่งทำหน้าที่ทั้งด้านประมวลผลความปลอดภัยและด้านการสื่อสารเครือข่าย ภายในบอร์ดมี **RTOS task** สองตัวที่แยกหน้าที่ชัดเจน ``SecTask`` ทำงานทุก 20 ms และถูกใช้เป็นลูปลหลักของระบบสำหรับ **drain remote command**, **poll keypad**, **poll ปุ่มจริง**, **poll sensor chain**, **ตรวจ timeout**, **drain `eventQueue`**, ประมวลผลผ่าน ``RuleEngine``, จากนั้นเรียก ``updateActuators()`` เพื่อขับฮาร์ดแวร์และจัดการ **state transition** แบบ **periodic** ส่วน ``MqttTask`` ทำงานทุก 10 ms เพื่อดูแล **Wi-Fi reconnect**, **MQTT reconnect**, **MQTT callback** และ **flush `publishQueue`**

จาก ``docs/layer_structure.md`` จะเห็นว่าการออกแบบนี้ตั้งใจแยก **task** ด้าน **network** ออกจาก **task** ด้าน **security** อย่างชัดเจน เพราะการเชื่อมต่อ **MQTT library** อาจมีพฤติกรรม **blocking** ระหว่าง **reconnect** ได้ หากปล่อยให้รันในลูปลเดียวกับ **keypad** และ **sensor** จะทำให้ระบบตอบสนองช้า การย้ายงานดังกล่าวไปไว้ใน ``MqttTask`` จึงช่วยให้ ``SecTask`` ยังคงทำงานตาม **cadence** ที่ตั้งไว้ได้ต่อเนื่อง

Local Interface & Access Hardware

อินเทอร์เฟซฝั่งผู้ใช้ภายในบ้านประกอบด้วย keypad แบบ `4x4 Matrix Membrane Keypad` (#27899) ที่ต่อผ่าน `PCF8574 I2C Expander`, จอ `SSD1306 OLED Display` และปุ่มจริงสำหรับ door/window lock-unlock toggle keypad รองรับการใส่รหัสผ่าน, การลบตัวอักษรทีละตัวผ่านปุ่ม `\*`, การล้างทั้ง buffer ผ่านปุ่ม `C`, การ submit ผ่าน `#`, การ mute warning ผ่านปุ่ม `A` และการขอความช่วยเหลือผ่านปุ่ม `B` ส่วน OLED ใช้แสดงผลของการป้อน PIN สถานะ door session หรือ hint การนับถอยหลังที่เกี่ยวข้อง

นอกจาก keypad แล้ว ระบบยังมีปุ่มจริงสองตัว ได้แก่ `BTN\_D` บน `GPIO33` และ `BTN\_W` บน `GPIO18` เพื่อให้ผู้ใช้สั่ง lock-unlock ประตูและหน้าต่างในระดับฮาร์ดแวร์ได้โดยตรง ปุ่มทั้งสองถูกอ่านผ่าน `EventCollector` พร้อม debounce และถูกแปลงเป็น `manual\_door\_toggle` หรือ `manual\_window\_toggle` ก่อนส่งไปยัง `SystemContext` ให้จัดการตรรกะต่อไป รายละเอียดพินของปุ่มถูกระบุไว้ใน `docs/pinallocation\_table.md`

Hardware Design

Main controller: ใช้ `ESP32-WROOM-32` พร้อม Wi-Fi และ Bluetooth เป็นศูนย์กลางการประมวลผลและสื่อสารทั้งหมด

Door reed switch: `Y213` ต่อที่ `GPIO32` ใช้ตรวจสอบสถานะเปิด-ปิดของประตู

Window reed switch: `Y213` ต่อที่ `GPIO19` ใช้ตรวจสอบสถานะเปิด-ปิดของหน้าต่าง

PIR 1: `HC-SR501` ต่อที่ `GPIO35` ใช้ตรวจการเคลื่อนไหวในโซนภายในชุดที่ 1

PIR 2: `HC-SR501` ต่อที่ `GPIO36` ใช้ตรวจการเคลื่อนไหวในโซนภายในชุดที่ 2

PIR Window zone: `HC-SR501` ต่อที่ `GPIO39` ใช้ตรวจการเคลื่อนไหวบริเวณหน้าต่างหรือ perimeter side

Tamper sensor: `SW-1801P` ต่อที่ `GPIO34` ใช้เป็นตัวชี้วัดการรบกวนหรือแรงกระแทกผิดปกติ

Ultrasonic set 1: `HC-SR04` ใช้ `GPIO13` เป็น TRIG และ `GPIO14` เป็น ECHO

Ultrasonic set 2: `HC-SR04` ใช้ `GPIO16` เป็น TRIG และ `GPIO17` เป็น ECHO

Ultrasonic set 3: `HC-SR04` ใช้ `GPIO4` เป็น TRIG และ `GPIO5` เป็น ECHO

Door toggle button: ปุ่มจริงสำหรับล็อกและปลดล็อกประตู ต่อที่ `GPIO33`

Window toggle button: ปุ่มจริงสำหรับล็อกและปลดล็อกหน้าต่าง ต่อที่ `GPIO18`

Door servo: `SG90` ต่อที่ `GPIO26` ใช้เป็น actuator สำหรับ lock mechanism ของประตู

Window servo: `SG90` ต่อที่ `GPIO27` ใช้เป็น actuator สำหรับ lock mechanism ของหน้าต่าง

Buzzer: piezo buzzer ต่อที่ `GPIO25` ใช้ส่งสัญญาณเสียงเตือนในระดับ Warn และ Alert

I2C bus: `GPIO21` เป็น SDA และ `GPIO22` เป็น SCL ใช้ร่วมกันระหว่าง `PCF8574` และ `SSD1306`

Keypad expander: `PCF8574` ใช้ที่ I2C address `0x20` สำหรับอ่าน matrix keypad

OLED display: `SSD1306` ใช้ที่ I2C address `0x3C` สำหรับแสดงผลสถานะระบบและ feedback จาก keypad

Operating Modes

Disarm Mode

โหมด `Disarm` เป็นโหมดพื้นฐานที่ถือว่าระบบไม่อยู่ในสถานะเฝ้าระวังการบุกรุกแบบเต็มรูปแบบ ผู้ใช้ที่ได้รับอนุญาตสามารถปลดล็อกประตูผ่าน keypad หรือคำสั่งระยะไกลได้ และยังเป็นฐานสำหรับการเข้าสู่ `Night` mode ด้วย เนื่องจาก policy ของระบบกำหนดว่า `arm\_night` ใช้ได้เมื่อ `latest\_mode == Disarm` เท่านั้น

แม้จะเป็นโหมดไม่เฝ้าระวังเต็มรูปแบบ แต่ `Disarm` ยังมีตรรกะสำคัญคือ auto-arm sequence โดย `chk1` จาก ultrasonic จะเริ่ม stage 1 จากนั้นเมื่อประตูเปิดจะเข้าสู่ stage 2 เมื่อประตูปิดจะเข้าสู่ stage 3 และหากไม่มี indoor/window activity ต่อเนื่อง 20 วินาที ระบบจะเปลี่ยนไป `Away` พร้อมล็อก actuator และบันทึกสถานะ

Away Mode

โหมด `Away` ใช้ในสถานการณ์ที่ไม่มีผู้อยู่อาศัยภายในบ้าน ระบบจะตรวจจับการบุกรุกแบบเต็มรูปแบบ และใช้ตรรกะไล่ระดับจาก `Off -> Warn -> Alert` หากมีการเปิดประตูโดยไม่มีหลักฐานการแจ้งเตือนก่อนหน้า ระบบจะเข้าสู่ `warn\_entry` พร้อมกำหนด keypad deadline 30 วินาทีเพื่อให้ผู้มีสิทธิ์สามารถปลดระบบได้ แต่หากมีแรงสั่นสะเทือนก่อนหน้า ประตูเปิดในขณะที่ยังล็อกอยู่ หรือหน้าต่างถูกเปิด ระบบสามารถข้าม grace period และเข้าสู่ `Alert` ได้ทันที

นอกจากเหตุการณ์ perimeter แล้ว โหมด Away ยังใช้ข้อมูลจาก PIR ภายในและ ultrasonic chokepoint เพื่อ step-up ระดับความเสี่ยงทีละขั้น เหตุการณ์อย่าง `motion 1`, `motion 2`, `motion 3`, `chk2` หรือ `chk3` จะถูกตีความว่าเป็นความเคลื่อนไหวผิดปกติภายในบ้านและทำให้ระดับความเสี่ยงขยับจาก `Off` ไป `Warn` และจาก `Warn` ไป `Alert` ตามลำดับ

Night Mode

โหมด `Night` ถูกออกแบบให้เป็น **perimeter-only protection** เพื่อให้ผู้อยู่อาศัยยังสามารถเคลื่อนไหวภายในบ้านได้ตามปกติในเวลากลางคืน แต่หากเกิดการเปิดประตู เปิดหน้าต่าง ตรวจพบแรงสั่นสะเทือน หรือมี **motion** จากโซน **perimeter** ระบบจะมองว่าเป็นการละเมิดขอบเขตและเข้าสู่ `alert\_night\_breach` ทันที

เหตุการณ์บางกลุ่ม เช่น `motion 1`, `motion 2`, `chk1`, `chk2` และ `chk3` ถูกกำหนดให้ **ignore** ในโหมด **Night** เพื่อป้องกัน **false alarm** จากการเคลื่อนไหวปกติของคนภายในบ้าน ตรรกะนี้ถูกระบุชัดใน `docs/possiblecase.md` และสะท้อนอยู่ใน `RuleEngine` ของระบบจริง

Software Design

โครงสร้างซอฟต์แวร์แบ่งเป็นชั้นตามหน้าที่เพื่อให้ควบคุมความซับซ้อนของระบบได้ เริ่มจาก configuration and shared types ซึ่งรวบรวม pin map, timing constant, enum และ queue payload structures จากนั้นเป็น hardware abstraction layer สำหรับติดต่อเซนเซอร์และ actuator โดยตรง core services จัดการ MQTT และ NVS ส่วน application logic layer รับผิดชอบการแปลง input เป็น event, ตีความ event ตาม mode และจัดการ state กลางของระบบ

เอกสาร `docs/layer_structure.md` แสดงให้เห็นว่าระบบไม่ปล่อยให้ hardware class ตัดสินใจเชิงนโยบายด้านความปลอดภัยโดยตรง เช่น `Sensors` อ่านค่าอย่างเดียว `Actuators` ขับอุปกรณ์อย่างเดียว ส่วนการตัดสินใจเหตุการณ์ใดควรถือเป็น Warn, Alert, หรือควรเปลี่ยน mode จะถูกเก็บไว้ใน `RuleEngine` และ `SystemContext` ส่งผลให้ตรรกะหลักรวมศูนย์และตรวจสอบได้ง่ายกว่า

Data Flow Design

การไหลของข้อมูลภายในระบบเริ่มจากการแยก event ตามแหล่งกำเนิด Remote command จาก MQTT จะถูก callback ฟังก์ชัน `MqttTask` รับเข้ามาและ enqueue ลง `commandQueue` ก่อน จากนั้น `SecTask` จึงหยิบมาประมวลผลในรอบของตัวเอง ส่วน local event จาก keypad, ปุ่มจริง และ sensor จะถูกสร้างผ่าน `EventCollector` แล้วส่งเข้า `eventQueue` เพื่อให้ `RuleEngine` drain ตามลำดับนี้ทำให้ command กับ event ไม่ปะปนกันโดยตรง แต่เชื่อมกันผ่าน queue ที่กำหนดหน้าที่ชัดเจน

สำหรับลำดับการ poll ภายใน `SecTask` ระบบกำหนด fixed order เป็น keypad ก่อน ตามด้วย local lock-unlock toggle buttons และค่อยเข้าสู่ sensor chain ซึ่งประกอบด้วย reed, PIR, vibration และ ultrasonic แบบ round-robin การจัดลำดับเช่นนี้ช่วยให้พฤติกรรมมีความแน่นอนและอธิบายได้จาก flowchart รวมทั้งลดโอกาสที่ input บางชนิดจะเบียดการทำงานของ input ที่สำคัญกว่าใน tick เดียวกัน

หลังจาก RuleEngine ประมวลผล event แล้ว `SystemContext` จะเป็นผู้ถือ state กลางและตัดสินใจต่อว่าจะ queue status, queue ack, บันทึก NVS, เริ่มหรือยุติ door session, update buzzer, หรือสั่ง servo ล็อก-ปลดล็อก เมื่อถึงช่วง periodic update ฟังก์ชัน `updateActuators()` ยังทำหน้าที่ดู auto-

arm stage timeout reset, เปลี่ยน stage 2 ไป stage 3, เปลี่ยน stage 3 ไป `Away`, และจัดการ
`doorSession\_.check()` เพื่อสร้าง `auto\_locked` หรือ `auto\_locked\_timeout` ตามกรณี

System Architecture

1. Sensing Layer

ชั้น **sensing** ประกอบด้วยอุปกรณ์รับสัญญาณทั้งหมดที่ใช้ตัดสินสถานะความปลอดภัยของบ้าน ได้แก่ **reed switch** สำหรับบอกการเปิด-ปิดประตูและหน้าต่าง, **PIR** สำหรับ **motion zone**, **vibration sensor** สำหรับตรวจแรงกระแทกหรือการจัดแงะ และ **ultrasonic sensor** สำหรับ **chokepoint detection** ภายใน **`EventCollector`** ข้อมูลเหล่านี้ไม่ได้ถูกใช้ตรง ๆ แต่จะถูกแปลงเป็น **event** เช่น **`door\_open`**, **`window\_open`**, **`motion 1`**, **`vib\_spike`**, **`chk1`**, **`chk2`**, **`chk3`** เพื่อให้ **state machine** ประมวลผลได้ง่ายขึ้น

Ultrasonic ถูกออกแบบให้ **poll** แบบ **round-robin** และใช้ **timeout** เพื่อจำกัดเวลาบล็อกของ **`pulseIn()`** แนวคิดนี้ช่วยให้ **task** ไม่ค้างนานเกินไปจาก **ultrasonic sensor** ตัวใดตัวหนึ่ง และทำให้ระบบยังสามารถรักษาความถี่ของลูปได้ใกล้เคียงกับที่ออกแบบ

2. Processing & Control Layer

ชั้นประมวลผลหลักประกอบด้วย **`EventCollector`**, **`RuleEngine`**, **`SystemContext`**, **shared types** และ **runtime stats** โดย **`RuleEngine`** เป็นตัวกำหนดพฤติกรรมตามโหมด เช่น ใน **Disarm** จะสนใจ **auto-arm sequence** ใน **Away** จะสนใจ **grace period**, **forced entry** และ **step-up alert** ส่วนใน **Night** จะสนใจเฉพาะ **perimeter breach** ส่วน **`SystemContext`** เป็นเจ้าของ **state** กลางของระบบ เช่น **`mode`**, **`alarm level`**, **`latest\_mode`**, **`is\_night`**, **`exit\_stage`**, **`door\_locked`**, **`window\_locked`** และตัวแปรช่วงเวลาอื่น ๆ

ชั้นนี้ยังรวม **policy logic** สำหรับ **remote command** เช่น การยอมรับ **`arm\_night`**, การยอมรับ **`night\_off`**, การจัดการ **`status`**, **`silence`** และ **lock-unlock command** ตลอดจนการ **queue** ข้อมูลออกไปยัง **`publishQueue`** เพื่อส่งต่อไปให้ **bridge** หรือ **subscriber** อื่นผ่าน **MQTT**

3. Actuation Layer

ชั้น **actuation** ประกอบด้วย SG90 สองตัวและ piezo buzzer หนึ่งตัว Door servo และ window servo ถูกใช้อธิบายสถานะ lock mechanism ของประตูและหน้าต่าง ส่วน buzzer ใช้บอกระดับความรุนแรงของสถานะ Warn หรือ Alert การขับอุปกรณ์เหล่านี้ไม่ได้ถูกเรียกจากหลายจุดแบบกระจาย แต่ถูกรวมผ่าน `Actuators` และ `SystemContext` ทำให้รูปแบบการสั่งงานคงที่และติดตามได้

นอกจากนี้ระบบยังมี local physical override ผ่านปุ่มจริงสองตัว ซึ่งแม้จะดูเหมือนสั่ง actuator ได้ตรง แต่ในโค้ดจริงก็ยังคงถูกแปลงเป็น event และส่งผ่าน flow กลางของระบบก่อนเสมอ เพื่อให้การเปลี่ยนแปลงสถานะและการ publish ข้อมูลออกไปมีความครบถ้วนและสอดคล้องกัน

4. Cloud Communication Layer

ชั้น cloud communication ใช้ MQTT เป็นสัญญาณระหว่างบอร์ดกับระบบภายนอก โดยหัวข้อหลักคือ `esh/main/cmd`, `esh/main/event`, `esh/main/status`, `esh/main/ack` และ `esh/main/metrics` บอร์ดจะ subscribe คำสั่งจาก `cmd` และ publish สถานะหรือเหตุการณ์ไปยังหัวข้ออื่น ส่วน LINE bridge จะ subscribe ข้อมูลเหล่านี้เพื่อแปลงเป็นข้อความและ bubble command ใน LINE OA

ในภาคใช้งานจริง bridge ทำงานร่วมกับ ngrok และ LINE webhook เพื่อเปิด endpoint ให้ LINE Messaging API ส่ง event เข้ามา จากนั้น bridge จึงแปลงข้อความหรือ postback เป็น MQTT command เช่น `status`, `silence`, `lock door`, `unlock all`, `arm\_night` และ `night\_off` การเชื่อมต่อส่วนนอกนี้ถูกออกแบบให้ไม่แตะ state ของบอร์ดตรง ๆ แต่สื่อสารผ่าน topic contract เท่านั้น

Operating Modes (State-Based Control)

1. Disarm Mode

ในมุมมองของ state machine โหมด 'Disarm' มีความสำคัญมากกว่าโหมดที่ไม่เตือนภัย เพราะมันเป็นจุดอ้างอิงสำหรับ 'latest_mode' และเป็นฐานสำหรับการเข้าออก Night mode ด้วย หากผู้ใช้ปลดระบบด้วย PIN ถูกต้อง 'SystemContext' จะ reset failed attempts, clear 'is_night', unlock door และเริ่ม door session ใหม่ แต่ถ้าอยู่ใน Disarm และเกิด sequence สำหรับ auto-arm ระบบจะเริ่มจัดเก็บ 'exit_stage' เพื่อรอว่าผู้ใช้ออกจากบ้านครบตาม pattern หรือไม่

ตาม 'possiblecase.md' ระบบจะเริ่ม stage 1 เมื่อเจอ 'chk1', stage 2 เมื่อเจอ 'door_open', stage 3 เมื่อประตูปิด และจะเปลี่ยนเป็น 'Away' หากไม่มี activity ที่ถือว่าขัดขวางภายใน 20 วินาที ในทางกลับกัน ถ้าระหว่าง stage 3 มี 'motion 1', 'motion 2', 'motion 3', 'chk2' หรือ 'chk3' ระบบจะ cancel auto-arm และ reset กลับไป stage 0 นอกจากนี้ stage 1 และ stage 2 ยังมี timeout reset 15 วินาทีเพื่อป้องกัน partial sequence ค้างคา

2. Away Mode

ใน Away mode state machine ให้ความสำคัญกับการแยกแยะระหว่าง authorized entry กับ forced entry หาก 'door_open' เกิดขึ้นโดยไม่มี vibration spike ล่าสุด ระบบจะเริ่ม entry deadline 30 วินาที ตั้งระดับเป็น Warn และเปิด warning buzzer แต่ถ้ามีหลักฐานเชิง tamper เช่น vibration spike ก่อนหน้า หรือประตูเปิดขณะที่ lock state ยังเป็น locked ระบบจะข้าม warn และเข้า Alert ได้ทันที

Away mode ยังมีพฤติกรรม step-up โดยอิง indoor motion และ chokepoint activity เช่น 'motion 1', 'motion 2', 'motion 3', 'chk2', 'chk3' ซึ่งทำให้ระดับจาก 'Off -> Warn -> Alert' ทีละขั้น ขณะที่ 'window_open' จะถูกมองว่าเป็น breach รุนแรงและไป 'alert_high' ได้ตรง ส่วน 'entry_timeout' ที่เกิดจากการหมดเวลา keypad grace period ก็จะไป 'alert_timeout' ทันทีเช่นกัน

3. Night Mode

Night mode ใช้ตรวจจับ perimeter-only ชัดเจนมาก เหตุการณ์ที่ถือเป็น breach คือ `door\_open`, `window\_open`, `vib\_spike` และ `motion 3` เท่านั้น เมื่อเกิดเหตุเหล่านี้ระบบจะตั้งระดับเป็น Alert และ queue เหตุผล `alert\_night\_breach` ออกไปใน MQTT เพื่อให้ bridge แจ้งเตือนทันที

ส่วนเหตุการณ์ภายในบ้าน เช่น `motion 1`, `motion 2`, `chk1`, `chk2` และ `chk3` จะถูก ignore เพื่อหลีกเลี่ยง false alarm เมื่อมีผู้อยู่อาศัยในบ้านตอนกลางคืน กลไกนี้ทำให้ Night mode เหมาะกับการใช้งานจริงมากกว่าการใช้ Away mode แทนในช่วงที่มีคนพักอาศัยอยู่

Data Flow Description

1. เมื่อบอร์ดเริ่มทำงาน ระบบจะ start serial, init watchdog, init `SystemContext`, init `EventCollector`, สร้าง `eventQueue` และสร้าง `SecTask` กับ `MqttTask`
2. NVS จะ restore ค่าของ `latest\_mode` และ `is\_night` เพื่อนำกลับมาสร้าง state หลักของระบบหลังรีสตาร์ท
3. `MqttTask` จะดูแลการเชื่อมต่อ Wi-Fi และ MQTT, รับ callback แล้ว enqueue ลง `commandQueue`, และ flush `publishQueue` ออกไปยัง broker
4. `SecTask` จะเริ่มแต่ละรอบด้วยการเช็ค `commandQueue` ก่อน โดยดึง remote command ได้สูงสุด 1 คำสั่งต่อ tick
5. หากไม่มี remote command ที่ต้องจัดการต่อ ระบบจะ poll local input ตามลำดับ keypad -> local buttons -> sensor chain
6. local event และ timeout event จะถูกส่งเข้า `eventQueue` ก่อน แล้ว `RuleEngine` จะ drain queue นี้เพื่อประมวลผลตาม mode
7. หลังจบการประมวลผล `SystemContext` จะ update actuator, transition auto-arm, door session และ queue status/event/ack ตามผลลัพธ์
8. bridge ภายนอกจะ subscribe topic เหล่านี้และแปลงเป็น LINE notification หรือ command result ให้ผู้ใช้

Functional Requirements

FR-1 ระบบต้องรองรับโหมด `Disarm`, `Away` และ `Night`

FR-2 ระบบต้องตรวจจับการเปิด-ปิดประตูและหน้าต่างด้วย Y213 reed switch

FR-3 ระบบต้องตรวจจับการเคลื่อนไหวด้วย HC-SR501 PIR ทั้งสามโซน

FR-4 ระบบต้องตรวจจับแรงสั่นสะเทือนหรือการรบกวนด้วย SW-1801P

FR-5 ระบบต้องตรวจจับ chokepoint activity ด้วย HC-SR04 จำนวน 3 ชุด

FR-6 ระบบต้องรองรับการป้อน PIN, help request และ silence command ผ่าน keypad

FR-7 ระบบต้องรองรับปุ่มจริงสำหรับ lock-unlock ประตูและหน้าต่าง

FR-8 ระบบต้องขับ SG90 door servo, SG90 window servo และ piezo buzzer ได้ตาม state

ของระบบ

FR-9 ระบบต้อง publish event, status, ack และ metrics ผ่าน MQTT topic ที่กำหนด

FR-10 ระบบต้องรองรับ remote command ผ่าน LINE OA และ bridge สำหรับ status, silence, lock-unlock และ night mode control

FR-11 ระบบต้องกู้คืนค่า `latest\_mode` และ `is\_night` จาก NVS ได้หลังรีสตาร์ท

Non-Functional Requirements

NFR-1 รูปความปลอดภัยหลักต้องตอบสนองต่อ input ได้สม่ำเสมอภายใต้ `SecTask` ที่กำหนดไว้ทุก 20 ms

NFR-2 งานด้าน MQTT reconnect ต้องไม่บล็อก keypad, local buttons และ sensor polling

NFR-3 โค้ดต้องแยกชั้น hardware, logic และ communication อย่างชัดเจนเพื่อให้ง่ายต่อการตรวจสอบและบำรุงรักษา

NFR-4 policy ของ remote command ต้องถูกบังคับที่ระดับ `SystemContext` หรือ flow กลาง ไม่ใช่ปล่อยให้ bridge ตัดสินใจเพียงฝั่งเดียว

NFR-5 ระบบต้องสามารถกู้คืนสถานะสำคัญหลัง power loss ได้

NFR-6 ฮาร์ดแวร์ที่เลือกต้องเป็นโมดูลต้นทุนต่ำ หาได้ง่าย และเหมาะกับการสร้างต้นแบบสำหรับการสาธิต

Software Components

1. `Config.h`, `Types.h`, `RuntimeStats.h` สำหรับกำหนดค่าพิน ค่าคงที่ timing enum และสถิติ runtime

2. `Sensors`, `Actuators`, `KeypadController`, `DisplayManager` เป็น hardware abstraction layer

3. `MqttService` และ `NvsStorage` เป็น core services สำหรับเครือข่ายและ persistence

4. `EventCollector`, `RuleEngine`, `SystemContext` เป็น application logic layer หลักของระบบ

5. `tools/line\_bridge` ทำหน้าที่เป็น external integration layer สำหรับ LINE OA, ngrok, launcher UI และ webhook

Development Methodology

กระบวนการพัฒนาของโครงการเป็นแบบ **iterative** และ **pragmatic** โดยเริ่มจากกำหนดขอบเขตพฤติกรรมที่ต้องเกิดจริงในระบบ แล้วค่อยแตกออกเป็น **layer**, **queue**, **task** และ **use case** จากนั้นนำไปผูกกับฮาร์ดแวร์จริงทีละชุด การพัฒนาในแต่ละรอบไม่ได้เน้นเพียงให้ **compile** ผ่าน แต่เน้นให้ **flowchart**, **possible cases** และโค้ดสอดคล้องกันด้วย เพื่อให้สามารถอธิบายระบบได้ทั้งเชิง **implementation** และเชิงเอกสาร

Requirement Analysis

ในช่วงวิเคราะห์ **requirement** ที่มิได้ปรับ **scope** จากแนวคิด **adaptive control** แบบกว้าง ให้เหลือระบบ **home security** ที่มีขอบเขตชัดและส่งมอบได้จริง **requirement** หลักจึงมุ่งไปที่ **intrusion monitoring**, **access control**, **alert escalation**, **local override** และ **remote supervision** มากกว่าการควบคุมสิ่งแวดล้อมภายในบ้าน ทั้งหมดนี้ถูกถอดออกมาเป็น **possible cases** ที่ตรวจสอบได้เป็นรายการ

System Design

การออกแบบระบบยึดเอกสาร ``blockdiagram``, ``flowchart``, ``layer_structure``, ``pinallocation`` และ ``possiblecase`` เป็นแกน โดยพยายามให้แต่ละเอกสารสะท้อนของจริง ไม่ใช่เป็นเพียง **conceptual diagram** จึงมีการระบุพินจริงของบอร์ด, **task** จริงของ **RTOS**, **queue** จริงในระบบ, **policy** จริงของ **remote command** และ **timeout** จริงของ **auto-arm/door session** ไว้อย่างชัดเจน

Implementation

การ **implement** ใช้ **PlatformIO** และ **C++** บน **ESP32** ฝัง **firmware** แยก **source** ตาม **layer** ส่วน **bridge** ใช้ **Python** เพื่อเชื่อม **LINE OA**, **MQTT** และ **UI launcher** ขั้นตอน **implementation** ที่สำคัญคือการแยก ``MqttTask`` ออกจาก ``SecTask``, การเพิ่ม **local physical toggle buttons**, การเพิ่ม **timeout reset** สำหรับ **auto-arm stage 1/2** และการปรับ **publish policy** ให้ **event** ไม่ **retain** เพื่อลดปัญหาแจ้งเตือนซ้ำหลัง **reconnect**

Integration

การรวมระบบครอบคลุมทั้งการ **wiring** ตาม **pin allocation**, การทดสอบการอ่าน **sensor**, การตรวจพฤติกรรมของ **servo** และ **buzzer**, การตรวจ **topic contract** ของ **MQTT** และการทดสอบ **end-to-end** ระหว่างบอร์ด, **broker**, **bridge**, **ngrok** และ **LINE OA** ในรอบ **integration** ยังมีการตรวจสอบความสอดคล้องระหว่าง **flowchart** กับโค้ดจริงอย่างต่อเนื่อง เพื่อให้ **behavior** ที่ใช้สอดคล้องตรงกับเอกสาร

Testing

การทดสอบของโครงการนี้เน้นการยืนยัน **behavior** ที่จำเป็นต่อการสาธิตและส่งงาน ได้แก่ การปลดล็อกผ่าน **PIN**, การแจ้งเตือนจากการบุกรุก, **auto-arm**, **night mode**, **local buttons**, **remote commands** และการ **reconnect** ของ **MQTT** โดยที่ลูปหลักไม่หยุด นอกจากการทดสอบบนฮาร์ดแวร์จริง ยังมีการ **review logic** แบบ **static** จากโค้ดเพื่อแก้ **defect** เชิงลำดับการทำงานที่อาจไม่เห็นจากเดโมสั้น ๆ เช่น **auto-arm stage** ค้างหรือ **retained event** บน **MQTT**

Testing Table

รหัสทดสอบ	สถานการณ์	ผลที่คาดหวัง	ผลลัพธ์
T1	ป้อน PIN ถูกต้องแล้วกด #	ระบบ เปลี่ยนเป็น Disarm, ปลดล็อกประตู, reset failed attempts และ publish mode_disarm	ผ่าน
T2	ป้อน PIN ผิดซ้ำ หลายครั้ง	ครั้งที่ 1-2 เป็น wrong_code และครั้งที่ 3 ขึ้นไปเป็น keypad_alert	ผ่าน
T3	ทำลำดับ auto- arm สำเร็จ	Disarm เปลี่ยนเป็น Away หลัง chk1, door open, door close และไม่มี activity 20 วินาที	ผ่าน
T4	ประตูเปิดใน Away โดยไม่ใส่ PIN ทันเวลา	เริ่ม warn_entry แล้ว escalate เป็น alert_timeout เมื่อหมด เวลา	ผ่าน
T5	เกิด perimeter breach ใน Night	door/window/ vibration/PIR3 ทำให้ เกิด alert_night_breach ทันที	ผ่าน

T6	ตัด MQTT ชั่วคราวระหว่างระบบ ทำงาน	MqttTask พยายาม reconnect ต่อไปโดย SecTask ยัง ตอบสนองต่อ keypad และ sensor ได้	ผ่าน
----	--	---	------

Use Case Diagram and Descriptions

Actors

1. Resident / Local User: ผู้ใช้ภายในบ้านที่ได้ตอบผ่าน keypad, OLED และปุ่มล็อกปลดล็อกจริง
2. Remote User: ผู้ใช้ภายนอกที่สั่งงานผ่าน LINE OA ซึ่งถูก bridge แปลงเป็น MQTT command
3. Sensor Environment: สภาพแวดล้อมจริงที่สร้างเหตุการณ์ เช่น door_open, motion, vibration และ chokepoint trigger
4. LINE Bridge / MQTT Broker: โครงสร้างกลางที่รับส่งคำสั่ง สถานะ การแจ้งเตือน และ acknowledgement

UC-1 Change Operating Mode

Main Flow:

1. ระบบรับเหตุการณ์เปลี่ยนโหมดจาก keypad, auto-arm sequence หรือ remote command
2. `RuleEngine` และ `SystemContext` ตรวจสอบว่าเหตุการณ์นั้นถูกต้องตาม mode และ policy หรือไม่
3. หากผ่านเงื่อนไข ระบบจะเปลี่ยน mode, update `latest\_mode` หรือ `is\_night`, บันทึก NVS และ queue status
4. actuator จะถูกอัปเดตให้สอดคล้องกับ mode ใหม่ เช่น auto-lock เมื่อเข้าสู่ Away

Possible / Alternative Cases:

1. `arm\_night` ถูกปฏิเสธหาก `latest\_mode` ไม่ใช่ `Disarm`
2. `night\_off` ถูกปฏิเสธหาก mode ปัจจุบันไม่ใช่ `Night`
3. auto-arm ถูกยกเลิกหากมี indoor/window activity ระหว่าง `exit\_stage == 3`
4. auto-arm stage 1 หรือ 2 reset เองหากขึ้นถัดไปไม่เกิดภายใน 15 วินาที

UC-2 Keypad Access & Local Lock Control

Main Flow:

1. ผู้ใช้กด keypad เพื่อใส่ PIN หรือกดปุ่มจริงเพื่อ lock-unlock ประตูและหน้าต่าง
2. `KeypadController` หรือ `EventCollector` แปลง input ให้เป็น event มาตรฐาน
3. `SystemContext` ดำเนินการ unlock/lock, เริ่มหรือยุติ door session และ queue สถานะ

ออกไป

4. OLED และ buzzer จะสะท้อนผลลัพธ์ให้ผู้ใช้งานทราบตามกรณี

Possible / Alternative Cases:

1. PIN ถูกต้องทำให้ระบบ Disarm และปลดล็อกประตู
2. PIN ผิด 1-2 ครั้งเป็น `wrong\_code`, ตั้งแต่ครั้งที่ 3 เป็น `keypad\_alert`
3. ปุ่ม `A` ใช้ silence warning และปุ่ม `B` ใช้ส่ง `keypad\_help`
4. ปุ่มจริงที่ `GPIO33` และ `GPIO18` จะ toggle lock state พร้อม publish เหตุผลแบบ manual

UC-3 Sensor Monitoring & Event Collection

Main Flow:

1. `SecTask` poll local input ตามลำดับคั้งทีในทุค 20 ms tick
2. `EventCollector` สร้าง event จาก keypad, ปุ่มจริง, reed, PIR, vibration และ ultrasonic
3. event ที่ได้จะเข้า `eventQueue` และถูก `RuleEngine` drain ตามลำดับ
4. `RuleEngine` ตีความ event ตาม mode ปัจจุบันและส่งผลไปยัง `SystemContext`

Possible / Alternative Cases:

1. keypad edit เช่น `\*` หรือ `C` จะกิน tick นั้นและข้าม sensor polling ลีกลงไป
2. ultrasonic ใช้การสลับตรวจทีละตัวแบบ round-robin
3. serial monitor ใน build นี้ใช้ debug-only และไม่ inject security event เข้าระบบ

UC-4 Security Monitoring & Alert

Main Flow:

1. เมื่อระบบอยู่ใน **Away** หรือ **Night** เหตุการณ์จาก **sensor** จะถูกประเมินตาม **security policy** ของ mode นั้น
2. ระบบอาจเข้าสู่ **Warn** ก่อน หรือ **Alert** ทันที ขึ้นกับชนิด **event** และหลักฐานประกอบ
3. **buzzer** และ **outgoing MQTT event/status** จะถูก **update** ตามระดับความเสี่ยงที่เกิดขึ้น
4. **LINE bridge** จะนำข้อมูลที่ได้ไปแปลงเป็นข้อความแจ้งเตือนให้ผู้ใช้

Possible / Alternative Cases:

1. ประตูเปิดพร้อม **vibration** ล่าสุดทำให้เป็น **`alert\_forced\_entry`** ทันที
2. หน้าต่างเปิดใน **Away** ทำให้เป็น **`alert\_high`**
3. หมดเวลา **keypad grace period** ทำให้เป็น **`alert\_timeout`**
4. **perimeter breach** ใน **Night** ทำให้เป็น **`alert\_night\_breach`**

UC-5 Remote Control & Notification

Main Flow:

1. ผู้ใช้ส่งคำสั่งจาก **LINE OA** ผ่าน **bubble command panel** หรือข้อความคำสั่ง
2. **LINE bridge** แปลงคำสั่งเป็น **MQTT payload** แล้วส่งไปที่ **topic `esh/main/cmd`**
3. **`MqttTask`** รับคำสั่งและ **enqueue** ลง **`commandQueue`**, จากนั้น **`SecTask`** จึง **dequeue** ไปประมวลผล
4. ระบบ **queue ack/status/event** กลับออกไปผ่าน **MQTT** และ **bridge** ตอบผู้ใช้หรือแจ้งเตือนตามประเภทเหตุการณ์

Possible / Alternative Cases:

1. คำสั่งที่ไม่รองรับจะได้ **failed acknowledgement** โดยไม่เปลี่ยน **state**
2. **`status`** จะคืน **snapshot** ของ **mode, risk, door state** และ **window state**
3. แม้เครือข่ายหลุด **`SecTask`** ก็ยังทำงานต่อได้ เพราะ **reconnect** อยู่ใน **`MqttTask`**

Conclusion

ระบบ **Smart Home Adaptive Control System** เวอร์ชันปัจจุบันเป็น **embedded security prototype** ที่มีขอบเขตชัดเจนและตรงกับงานจริงในโปรเจกต์ โดยใช้ **`ESP32-WROOM-32`** เป็นแกนหลักและออกแบบให้รองรับทั้งการใช้งาน **local** และ **remote** ภายใต้ **state machine** ที่ชัดเจน จุดเด่นของระบบคือการประสานข้อมูลจากหลายเซนเซอร์เข้ากับ **mode-based logic**, การแยก **task** ด้าน **network** ออกจาก **task** ด้าน **security** และการมีเอกสารประกอบที่สอดคล้องกับโค้ดจริง

เมื่อพิจารณาจากโครงสร้างซอฟต์แวร์ ฮาร์ดแวร์ที่เลือกใช้ **use cases** และ **possible cases** ระบบนี้สามารถใช้เป็นต้นแบบสำหรับการสาธิตและประเมินแนวคิด **smart home security** ได้อย่างครบถ้วน ทั้งในด้านการตรวจจับการบุกรุก การปลดล็อกและล็อกอัตโนมัติ การแจ้งเตือนผ่าน **LINE OA** และการคงความตอบสนองของอุปกรณ์แม้ในสภาวะ **reconnect** ของเครือข่าย